Section Overview

- Applying the Roofline Model: Block Tiling and Thread Coarsening

Driving Question

How can we increase AI?

The roofline model tells us that increasing AI should increase throughput.

But how can we actually increase AI?

Driving Question

How can we increase AI?

The roofline model tells us that increasing AI should increase throughput.
But how can we actually increase AI?

1. Decrease DRAM usage by utilizing SMEM more aggressively.

Driving Question

How can we increase AI?

The roofline model tells us that increasing AI should increase throughput.
But how can we actually increase AI?

1. Decrease DRAM usage by utilizing SMEM more aggressively.
2. Loading larger tiles into SMEM.

Driving Question

How can we increase AI?

The roofline model tells us that increasing AI should increase throughput.
But how can we actually increase AI?

1. Decrease DRAM usage by utilizing SMEM more aggressively.
2. Loading larger tiles into SMEM.
3. Reusing values in SMEM to compute multiple values of C.

Generalized Tiling: Loading larger tiles.

Our tiled kernel **hardcoded** a square 32×32 block size. Each thread in the block load one value into SMEM. We can parameterize the tiling with BM, BN, BK.

Basic tiling: one thread, one element from each tile

```
__shared__ float As[BK][BK];
__shared__ float Bs[BK][BK];
float sum = 0.0;

for (int k = 0; k < K; k += BK) {

  const int A_tile_x = k + xt;
  if (y < M && A_tile_x < K) {
    As[yt][xt] = A[y * K + A_tile_x];
  } else {
    As[yt][xt] = 0.0;
  }

  const int B_tile_y = k + yt;
  if (B_tile_y < K && x < N) {
    Bs[yt][xt] = B[B_tile_y * N + x];
  } else {
    Bs[yt][xt] = 0.0;
  }
  __syncthreads();

  for (int i = 0; i < BK; i++) {
    sum += As[yt][i] * Bs[i][xt];
  }
  __syncthreads(); // Compute partial sum using As, Bs ...
```

Direct mapping

The thread coordinates directly select the shared-memory cells:

$$(y_t, x_t) \longrightarrow \begin{cases} A_s[y_t][x_t] \\ B_s[y_t][x_t] \end{cases}$$

Limitation

This only works because the block and both shared-memory tiles are all $BK \times BK$.

General tiling: threads cooperatively load each tile

```
const int tid = yt * BN + xt;
const int numThreads = BM * BN;
__shared__ float As[BM * BK];
__shared__ float Bs[BK * BN];

for (int k = 0; k < K; k += BK) {
  for (int id = tid; id < BM * BK; id += numThreads) {
    const int yid = id / BK, xid = id % BK;
    const int A_tile_y = yb + yid;
    const int A_tile_x = k + xid;

    if (A_tile_y < M && A_tile_x < K) {
      As[id] = A[A_tile_y * K + A_tile_x];
    } else {
      As[id] = 0.0;
    }
  }

  for (int id = tid; id < BK * BN; id += numThreads) {
    const int yid = id / BK, xid = id % BK;
    const int B_tile_y = k + yid;
    const int B_tile_x = xb + xid;

    if (B_tile_y < K && B_tile_x < N) {
      Bs[id] = B[B_tile_y * N + B_tile_x];
    } else {
      Bs[id] = 0.0;
    }
  }
  __syncthreads(); // Compute partial sum using As, Bs ...
```

Thread-strided loading

Flatten each tile. Thread *tid* loads

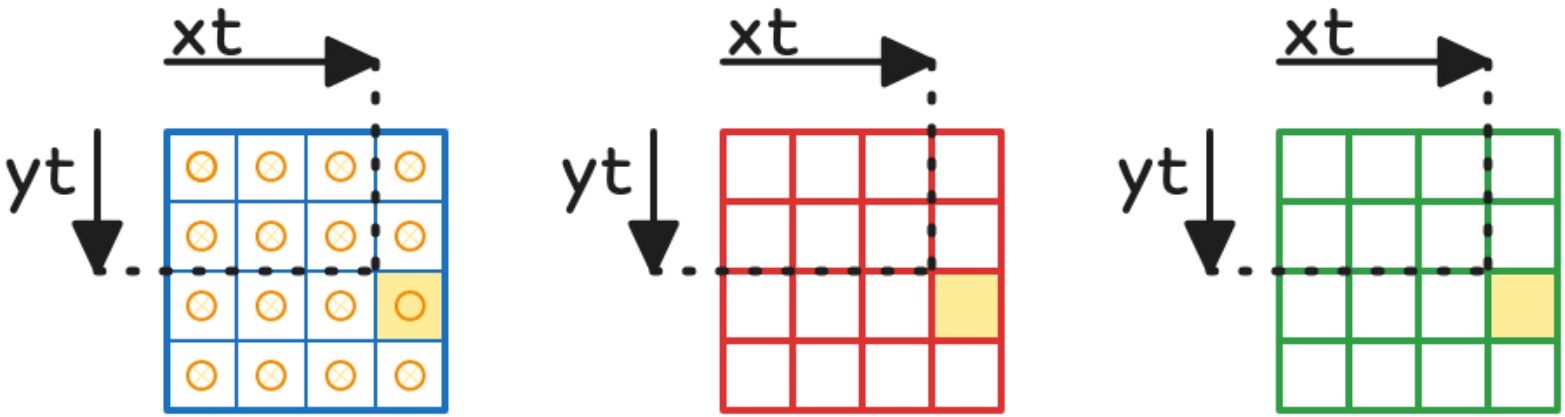
$$id = tid + jT$$

for $j = 0, 1, \ldots,$ where $T = BM \cdot BN$ is the thread count.

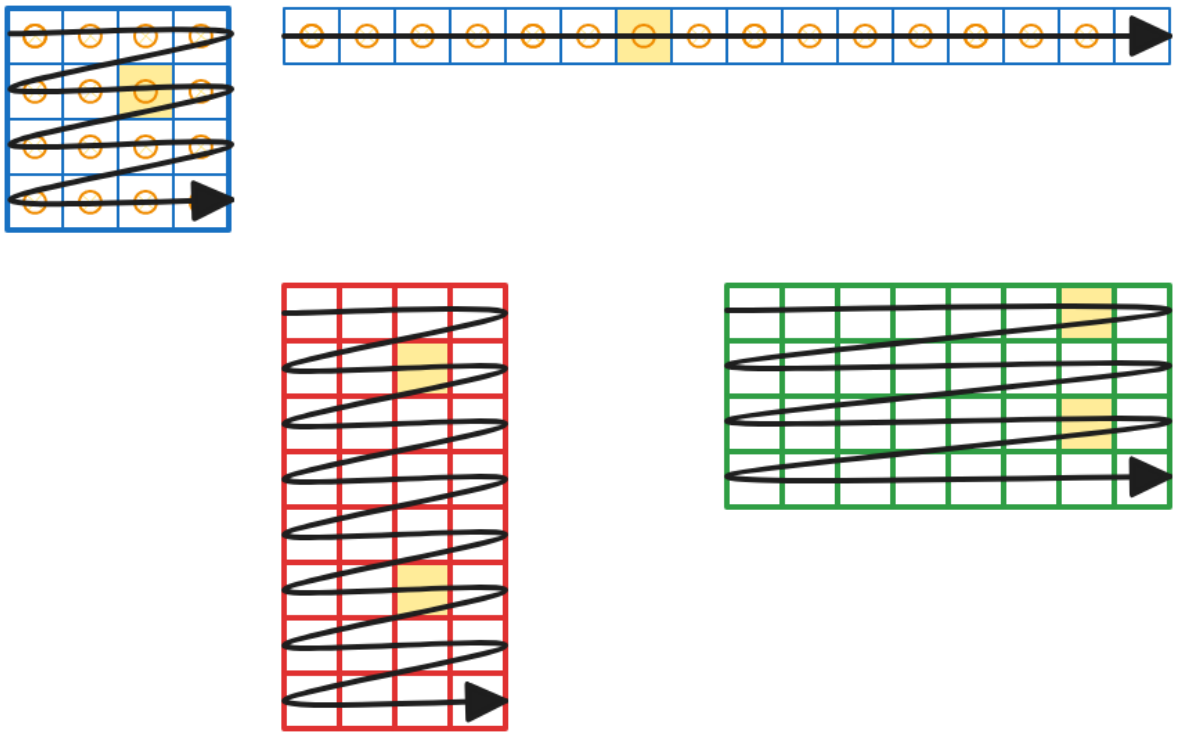
Key difference

A thread may load zero, one, or several values. Loading no longer assumes $BM = BN = BK$.

Visualization

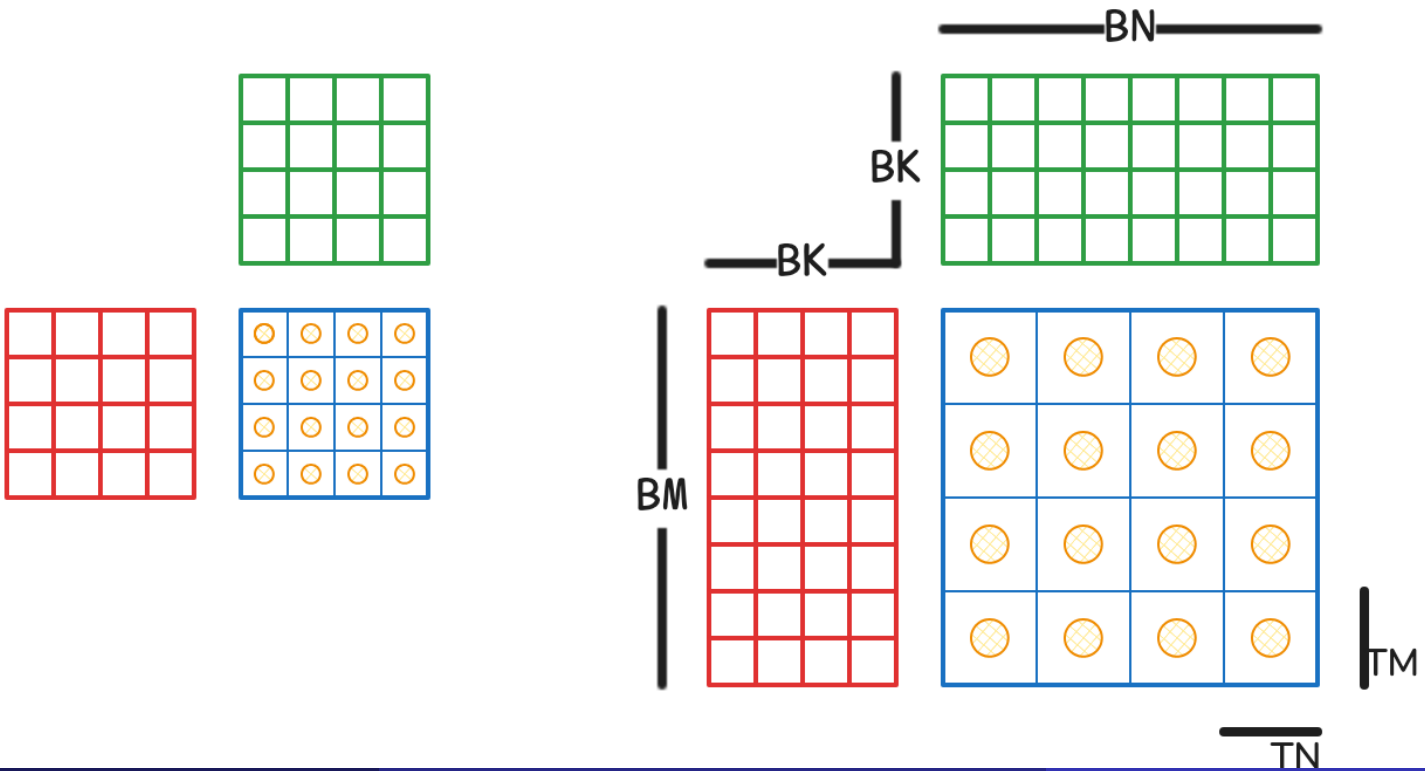


Visualization

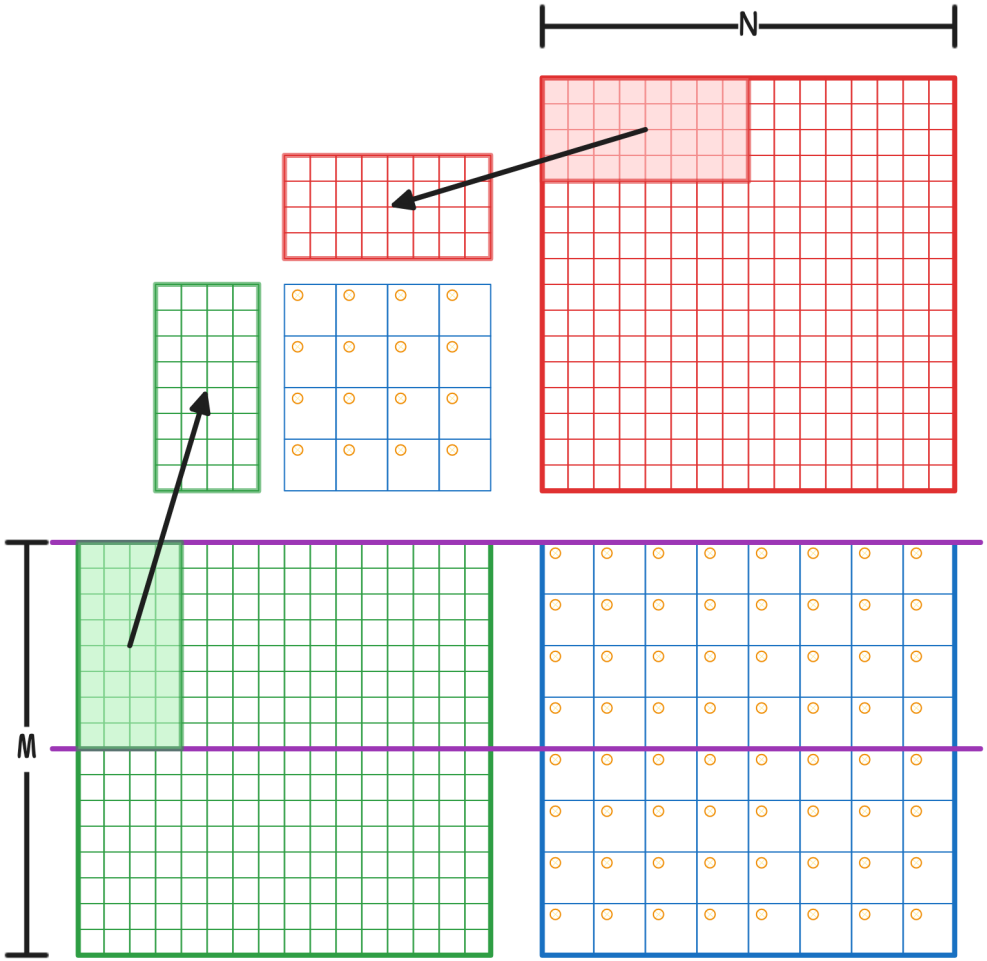


Thread coarsening: Increasing AI

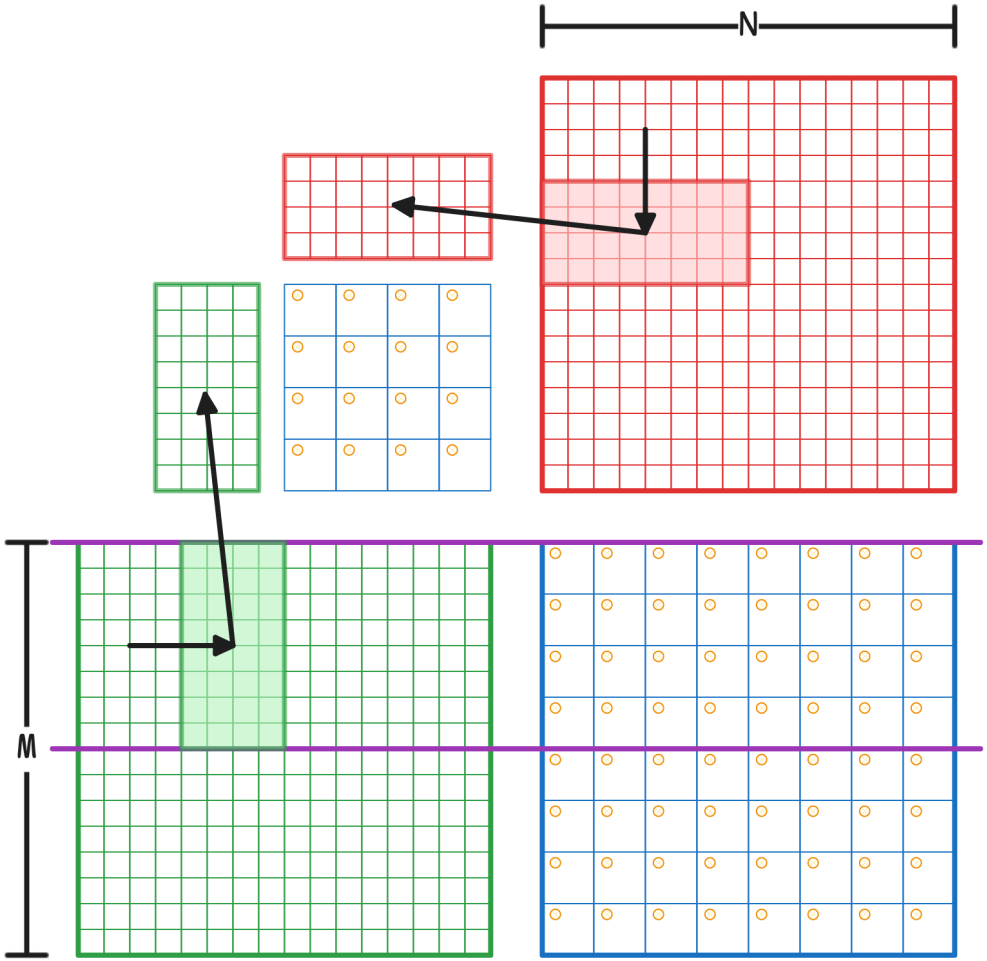
- Basic tiling assigns 1 element of the output to each thread.
- We can *coarsen* each thread, and assign a $TM \times TN$ portion of the output to each thread.
- We load larger chunks into SMEM, increasing data reuse and AI.
- One loaded A value can update several columns; one loaded B value can update several rows.



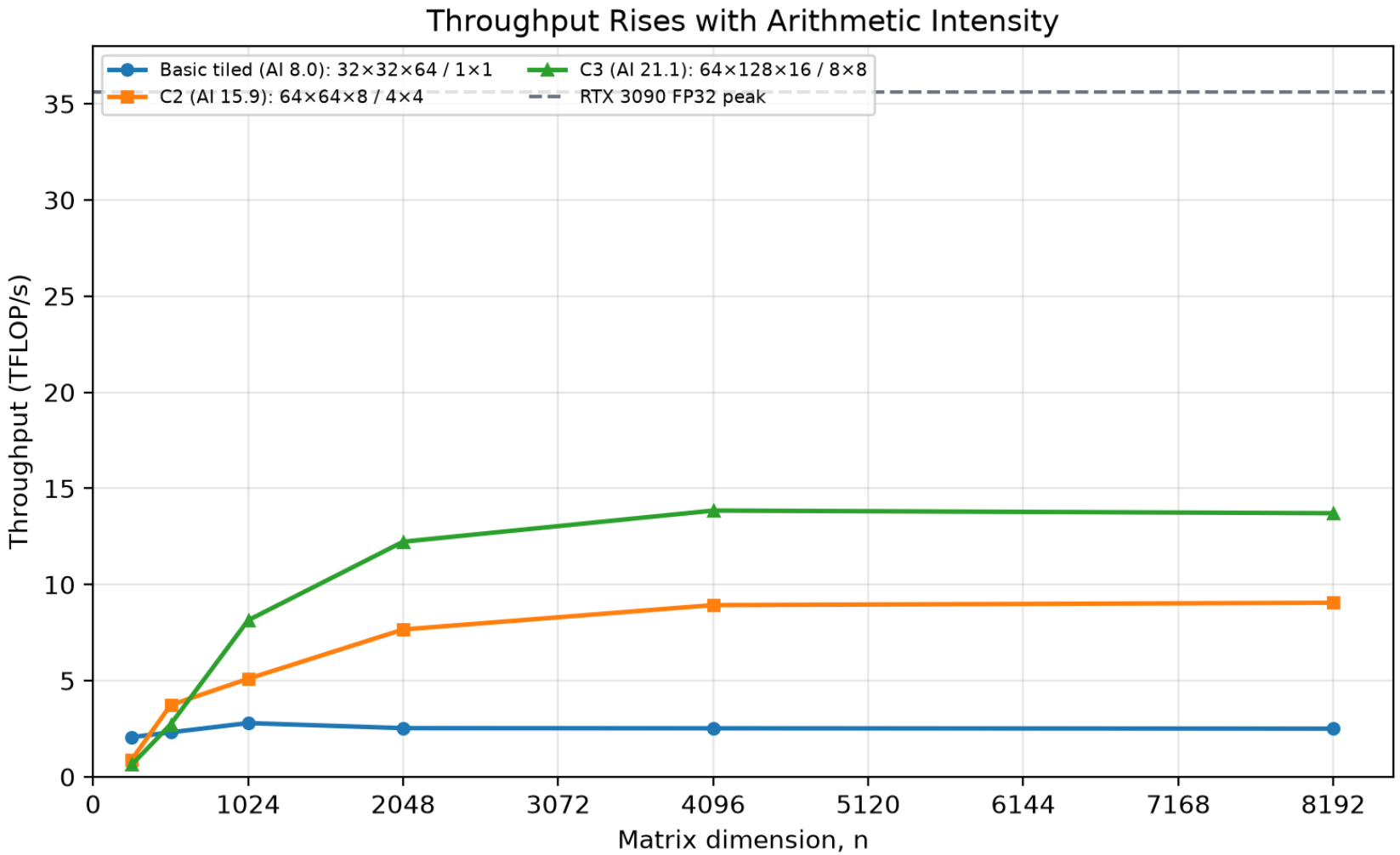
Coarsened SGEMM visualization



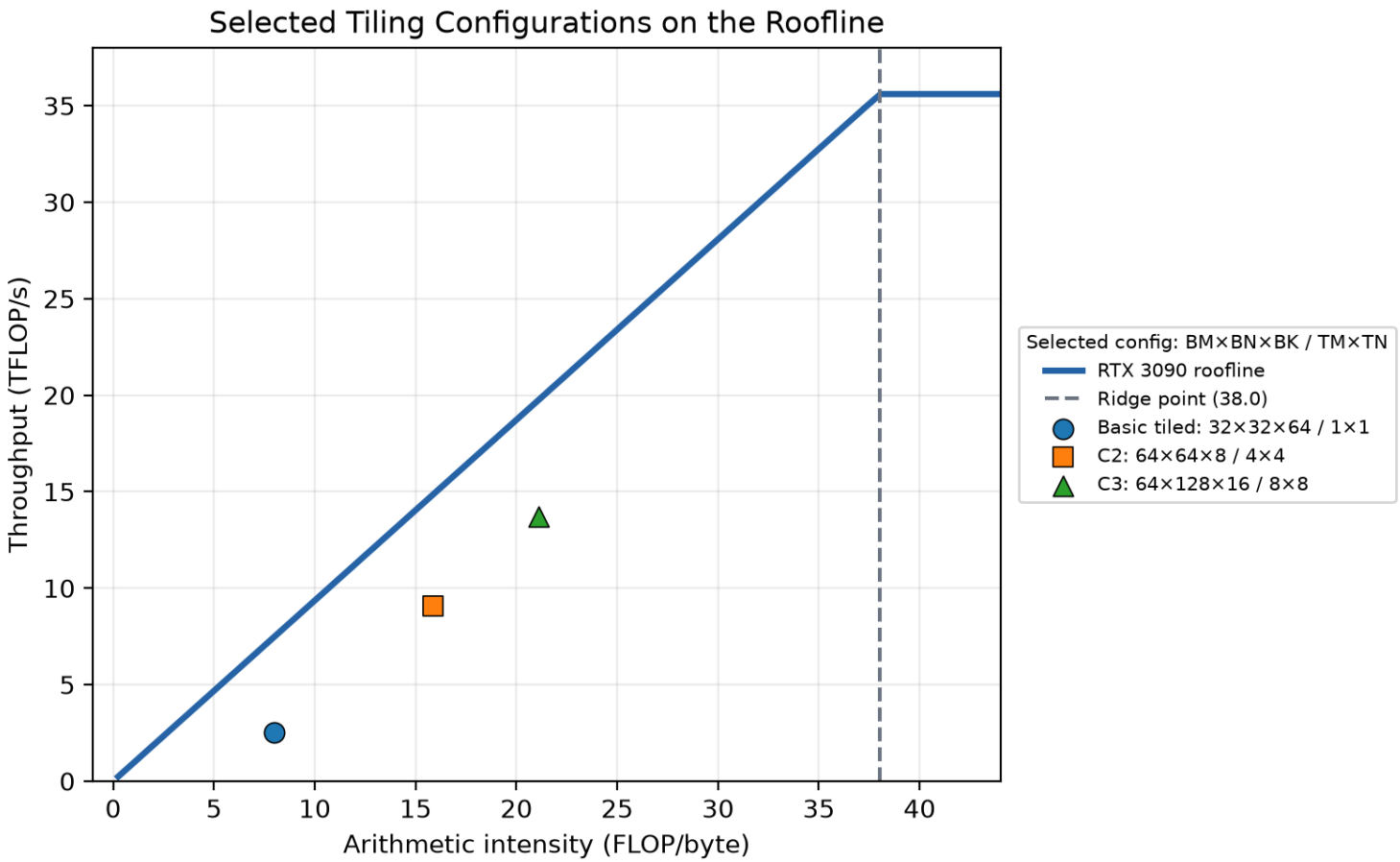
Coarsened SGEMM visualization



Higher-AI tiles tend to raise throughput

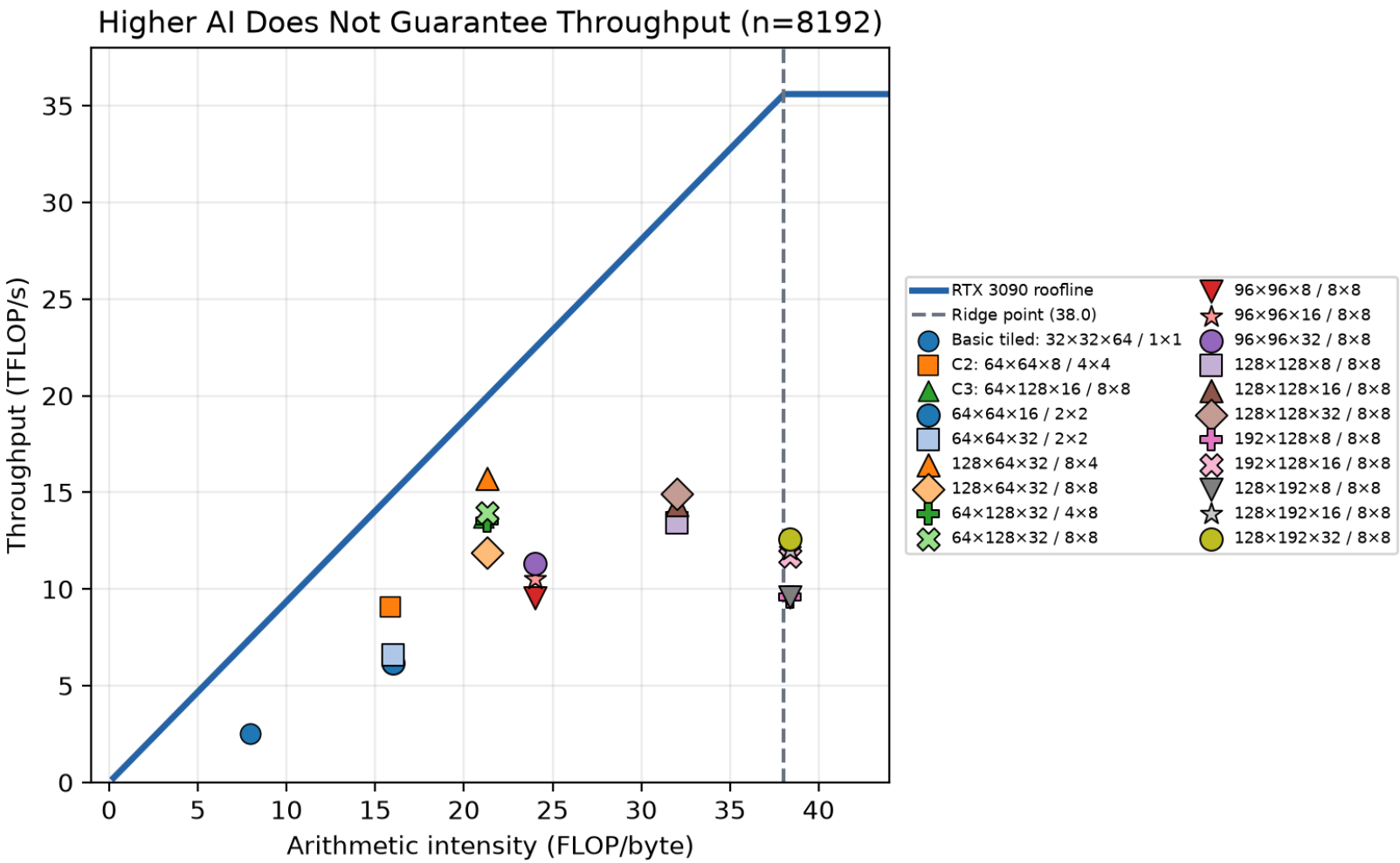


Tiling configurations on the roofline



Reminder that the Roofline Model only tells us the upper bound!

Higher AI does not guarantee higher throughput



The roofline is an upper bound, not a performance prediction.

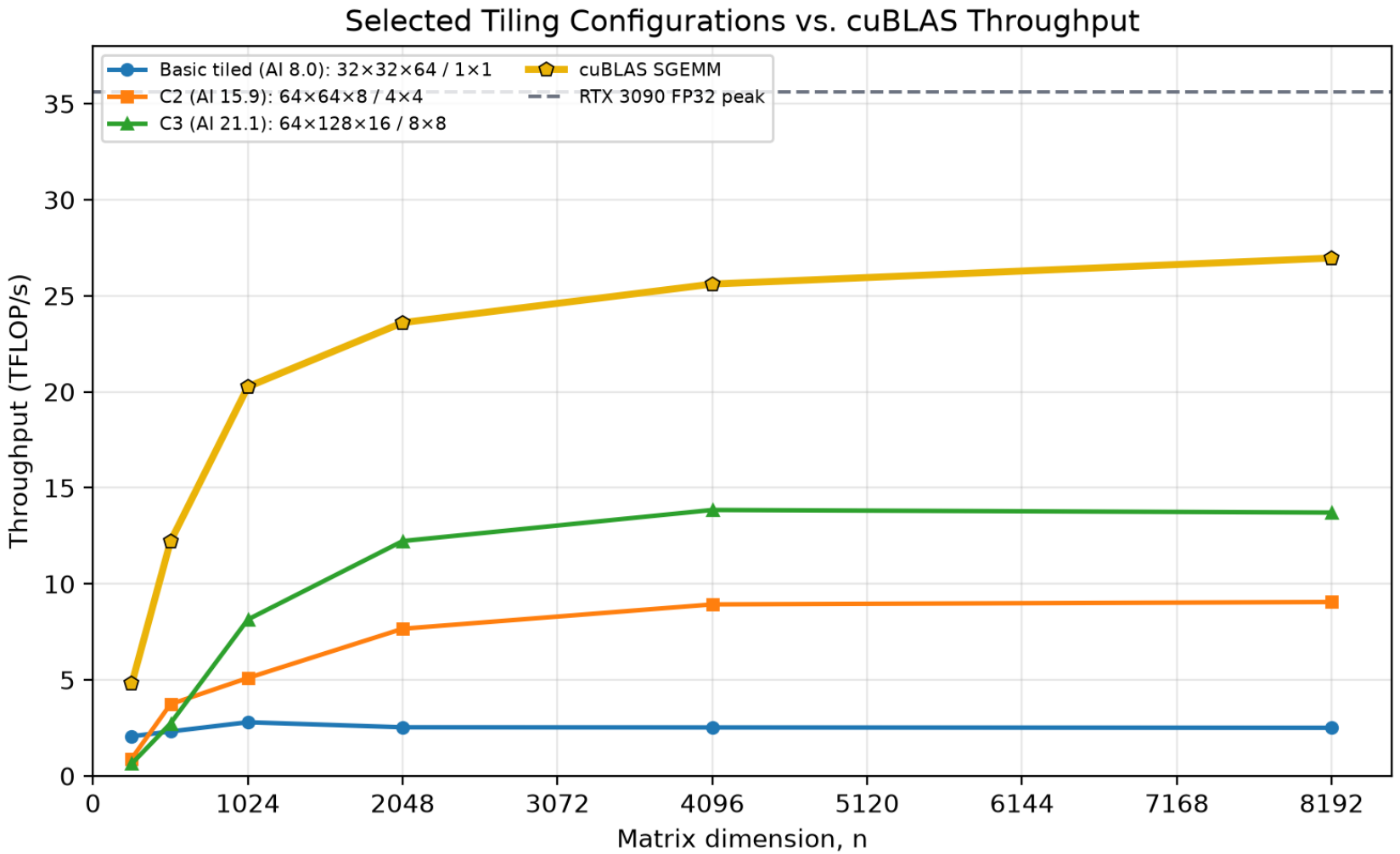
We have only scratched the surface: cuBLAS

- cuBLAS is NVIDIA's architecture-tuned linear algebra library and the practical performance target for our SGEMM kernels.
- At $n = 8192$, cuBLAS reaches 26.96 TFLOP/s, while the highest-AI selected configuration (C3) reaches 13.71 TFLOP/s, or 50.8% of cuBLAS's throughput.
- cuBLAS completes the multiplication in 40.8 ms versus 80.2 ms for C3, about 1.97× faster.

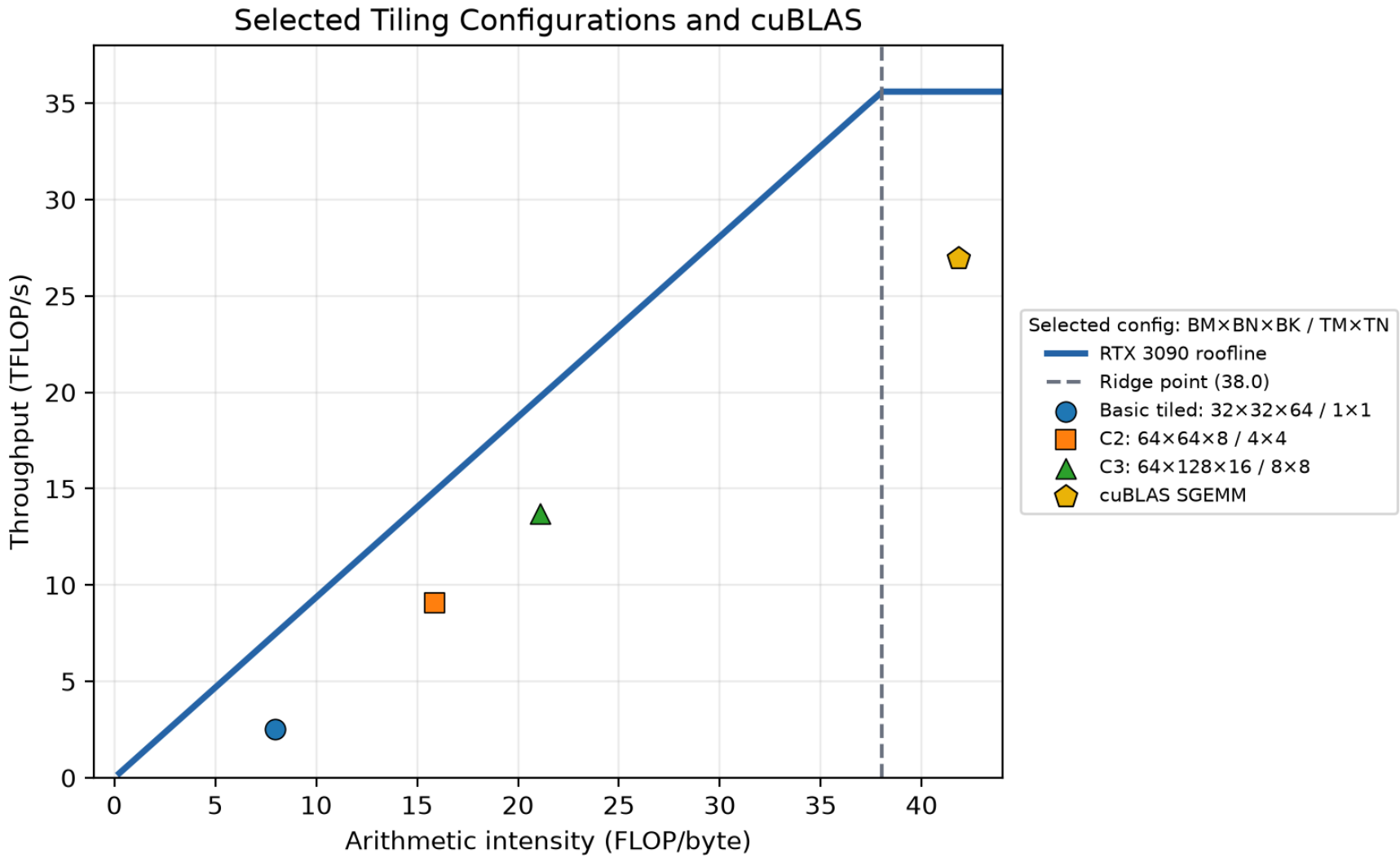
The remaining gap

Production kernels combine deeper register and warp tiling, vectorized memory access, pipelined loads, and architecture-specific instruction scheduling.

cuBLAS remains the performance target



cuBLAS on the roofline



cuBLAS arithmetic intensity is estimated from the profiled 256×128 output-tile shape.

We've come a long way

